《算法分析与设计》

课后作业

学　　号　2024112639

姓　　名　毛露涵

年　　级　2024 级

专　　业　软件工程

院　　系　计算机与人工智能学院

第一大题（50%）	第二大题（50%）	第三大题（0%）	第四大题（0%）	总分	批阅人签字

2025-2026 学年第 2 学期

第 4 章 分治法

一、编程题 1（共 50 分）

设 A[1]=”*”,　A[2]为如下的形状：

```
    *
  *   *
    *
```

也就是 A[2]为：

```
      A[1]
  A[1]   A[1]
      A[1]
```

对应的 A[n]为：

```
      A[n-1]
  A[n-1   A[n-1]
      A[n-1]
```

输入要求：

输入整数 n，占 1 行。

输出要求：

对应的图形。

样例输入：

3

样例输出：

```
        *
      *   *
        *
    *       *
  *   *   *   *
    *       *
        *
      *   *
        *
```

要求：

（1） 采用分治法编写程序实现上述题目要求，附源代码（30 分）。

源代码：

```
#include <bits/stdc++.h>

using namespace std;

vector<vector<char> > pic;

int getSize(int n)
{
	int s = 1;
	for (int i = 1; i < n; i++)
	{
		s *= 3;
	}
	return s;
}

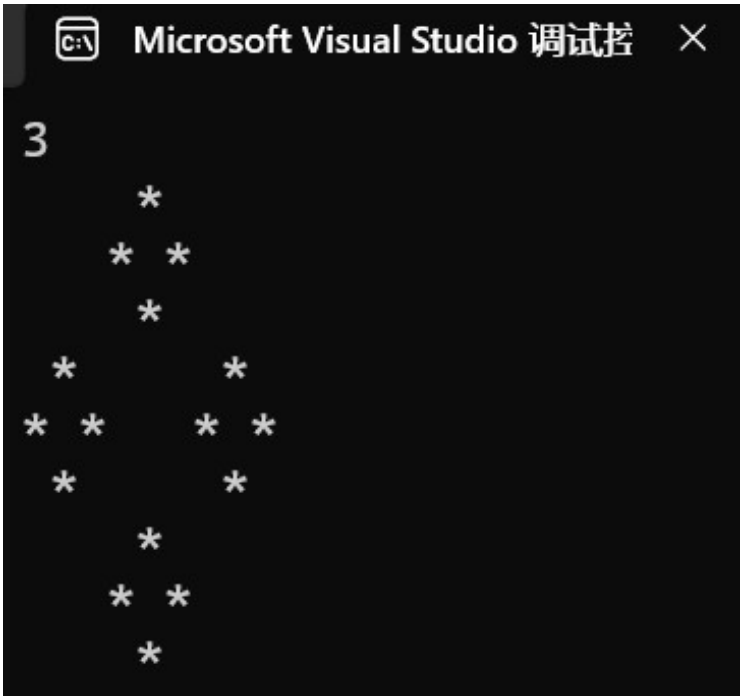
void draw(int n, int x, int y, int len)
{
	if (n == 1)
	{
		pic[x][y] = '*';
		return;
	}
	int sub = len / 3;
	draw(n - 1, x, y + sub, sub);
	draw(n - 1, x + sub, y, sub);
	draw(n - 1, x + sub, y + 2 * sub, sub);
	draw(n - 1, x + 2 * sub, y + sub, sub);
}

int main()
{
	int n;
```

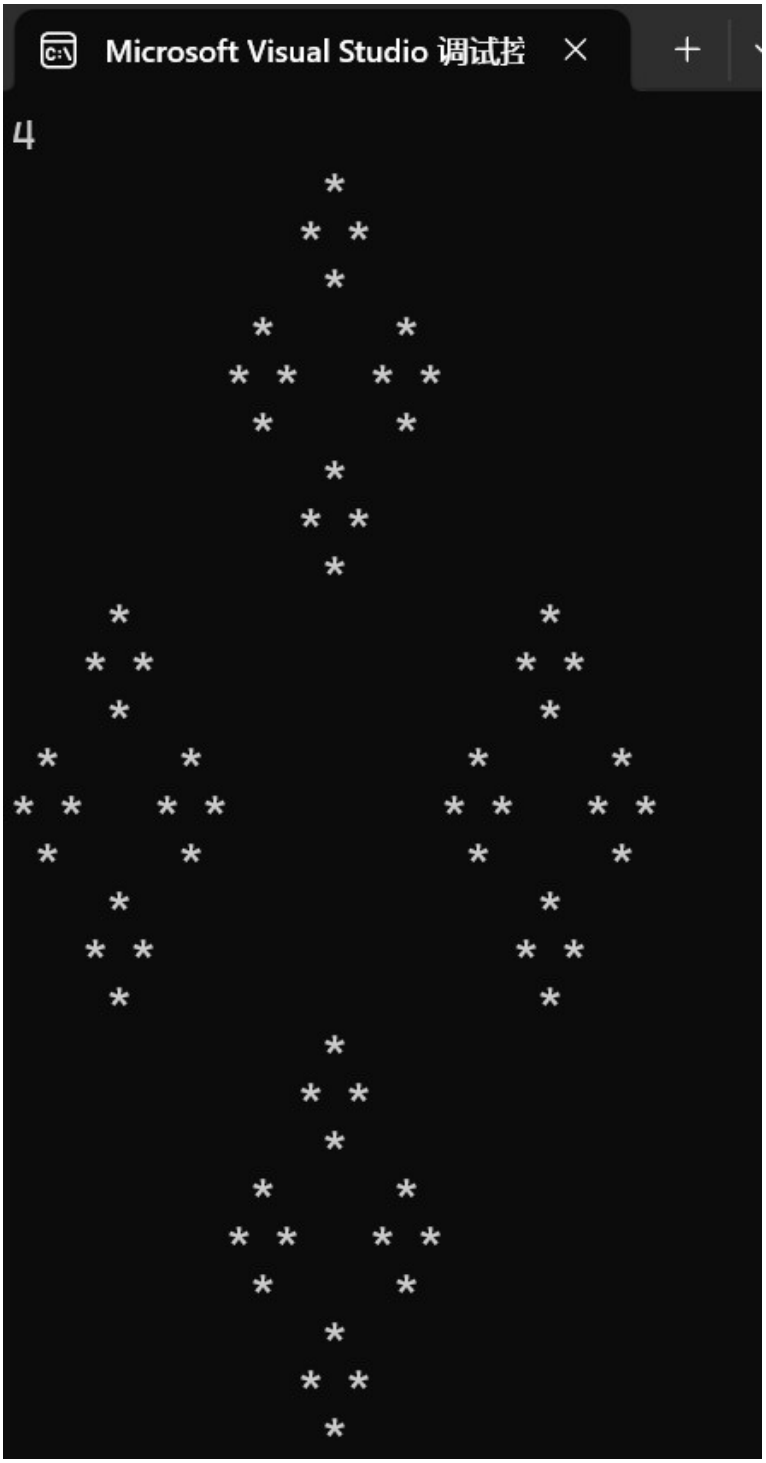
```
    cin >> n;
    int len = getSize(n);
    pic.assign(len, vector<char>(len, ' '));
    draw(n, 0, 0, len);
    for (int i = 0; i < len; i++)
    {
        int last = len - 1;
        while (last >= 0 && pic[i][last] == ' ')
        {
            last--;
        }
        for (int j = 0; j <= last; j++)
        {
            cout << pic[i][j];
        }
        cout << endl;
    }
    return 0;
}
```

（2） 给出 n=3 和 n=4 的软件测试屏幕截图（10 分）。

n=3 时



n=4 时



（3）分析算法的时间复杂性（10 分）。

设 A[n]图形的边长为 3^(n-1)，因此整个输出矩阵的规模为 3^(n-1)×3^(n-1)

递归绘制时，每个 A[n]由 4 个 A[n-1]组成，所以递归部分的时间复杂性递推式为 T(n)=4T(n-1)+O(1)。由此可得，递归绘制部分的时间复杂度为 O(4^(n-1))

但是程序最后需要输出整个二维矩阵，而矩阵规模为 9^(n-1)，输出操作的时间开销大于递归绘制部分。因此，该算法的总时间复杂度为 O(9^(n-1))

二、编程题 2（共 50 分）

某供电公司在某省各地修建了很多的变电站，这些变电站在出现故障时需要及时的维修，因此派遣距离这些变电站最近的工人去维修将会极大的减少维修的费用。

输入：

输入第一行包含一个整数 N ($1 \le N \le 100000$)，表示变电站和工人的数量。其后的 2N 行，每一行有两个整数 X ($0 \le X \le 1000000000$)，Y ($0 \le Y \le 1000000000$)，前面的 N 行表示变电站的位置坐标，后面的 N 行表示工人的位置坐标。

输出：

1 行，也就是与这些变电站距离最近的工人所对应的最小距离，精确到小数点后 3 位。

样例输入：

4
0 0
0 1
1 0
1 1
2 2
2 3
3 2
3 3

样例输出：

1.414

要求：

（1） 采用分治法编写程序实现上述题目要求，附源代码（30 分）。

源代码：

```
#include <bits/stdc++.h>

using namespace std;

struct Pt
{
    double x, y;
```

```
    int tp;
};

vector<Pt> a, b;

bool cmpx(Pt u, Pt v)
{
    if (u.x != v.x)
    {
        return u.x < v.x;
    }
    return u.y < v.y;
}

bool cmpy(Pt u, Pt v)
{
    return u.y < v.y;
}

double dis(Pt u, Pt v)
{
    if (u.tp == v.tp)
    {
        return 1e18;
    }
    double dx = u.x - v.x;
    double dy = u.y - v.y;
    return sqrt(dx * dx + dy * dy);
}

double dfs(int l, int r)
{
    if (r - l <= 3)
    {
```

```
        double res = 1e18;
        for (int i = l; i <= r; i++)
        {
            for (int j = i + 1; j <= r; j++)
            {
                res = min(res, dis(a[i], a[j]));
            }
        }
        sort(a.begin() + l, a.begin() + r + 1, cmpy);
        return res;
    }
    int m = (l + r) / 2;
    double mx = a[m].x;
    double d1 = dfs(l, m);
    double d2 = dfs(m + 1, r);
    double res = min(d1, d2);
    merge(a.begin() + l, a.begin() + m + 1,
        a.begin() + m + 1, a.begin() + r + 1,
        b.begin(), cmpy);
    for (int i = l, j = 0; i <= r; i++, j++)
    {
        a[i] = b[j];
    }
    int cnt = 0;
    for (int i = l; i <= r; i++)
    {
        if (fabs(a[i].x - mx) < res)
        {
            b[cnt++] = a[i];
        }
    }
    for (int i = 0; i < cnt; i++)
    {
        for (int j = i + 1; j < cnt && b[j].y - b[i].y < res; j++)
```

```
            {
                res = min(res, dis(b[i], b[j]));
            }
        }
        return res;
}

int main()
{
        int n;
        cin >> n;
        a.resize(2 * n);
        b.resize(2 * n);
        for (int i = 0; i < n; i++)
        {
            cin >> a[i].x >> a[i].y;
            a[i].tp = 0;
        }
        for (int i = n; i < 2 * n; i++)
        {
            cin >> a[i].x >> a[i].y;
            a[i].tp = 1;
        }
        sort(a.begin(), a.end(), cmpx);
        double res = dfs(0, 2 * n - 1);
        cout << fixed << setprecision(3) << res << endl;
        return 0;
}
```

（2）给出上述测试用例和如下测试用例的软件测试屏幕截图（10 分）。

```
5
11  45
14  19
19  810
19  26
```

8 17
21 23
70 13
1 37
95 7
1 1

样例一：



样例二：



（3）分析算法的时间复杂性（10 分）。

设变电站数量为 n，工人数量也为 n，因此程序中处理的总点数为 2n。

程序首先对所有点按照横坐标进行排序，排序的时间复杂度为 O(nlogn)。

在递归函数 dfs 中，每次将当前点集划分为左右两个规模大致相同的子集，并分别递归求解左右两部分的最小距离。合并时，程序会按照纵坐标合并当前区间中的点，并检查分界线附近可能产生更小距离的候选点。

每一层递归中，所有区间合起来处理的点数仍然是 2n，所以每一层的处理时间为 O(n)。由于每次递归都将问题规模约减半，因此递归层数为 O(logn)。

因此，递归求解部分的时间复杂度为 O(nlogn)。再加上初始排序的 O(nlogn)，整个算法的总时间复杂度为 O(nlogn)。其中 n 表示变电站或工人的数量。